Compiler Design Laboratory
Major Lab Assignment: LL(1) Parser Construction in C

---

1. Objective

The objective of this lab is to implement a complete LL(1) parser generator in C that:

- Removes left recursion
- Performs left factoring
- Computes FIRST and FOLLOW sets
- Constructs predictive parsing table
- Parses input strings using stack-based LL(1) parsing

All tasks must be implemented programmatically in C.

2. Input Grammar

Your program must accept grammar productions in the following format:

```
S->Sa|bSc|bSd|e
```

The grammar may contain:

- Immediate left recursion
- Common prefixes requiring left factoring

3. Task 1: Remove Left Recursion (In C)

Your program must:

- Detect immediate left recursion
- Apply the transformation:

$$A \to A\alpha \mid \beta \Rightarrow A \to \beta A'$$

$$A' \to \alpha A' \mid \epsilon$$

- Display the transformed grammar

4. Task 2: Perform Left Factoring (In C)

Your program must:

- Detect common prefixes
- Apply left factoring automatically
- Display the fully factored grammar

5. Task 3: Compute FIRST and FOLLOW Sets (In C)

Your program must:

- Compute FIRST for each non-terminal
- Compute FOLLOW for each non-terminal
- Handle epsilon correctly
- Display the computed sets

6. Task 4: Construct Predictive Parsing Table (In C)

Using FIRST and FOLLOW:

- Construct LL(1) parsing table
- Display the parsing table in matrix format
- Detect conflicts (if grammar is not LL(1))

7. Task 5: Implement LL(1) Parser (In C)

Your program must:

- Accept an input string ending with $
- Use stack-based predictive parsing
- Display:
  - Stack contents
  - Remaining input
  - Production used
- Announce:
  - "String Accepted" if parsing succeeds
  - "Syntax Error" if parsing fails

8. Required Output Format

Example:

```
After Removing Left Recursion:
...

After Left Factoring:
...

FIRST Sets:
FIRST(S) = { ... }

FOLLOW Sets:
FOLLOW(S) = { ... }
```

```
Predictive Parsing Table:
      a     b     c     $
S     ...   ...   ...

Parsing Trace:
Stack       Input         Production
-----------------------------------------
S$          b e c a$      S -> ...
...

String Accepted
```

9. Detailed C Program Skeleton

Students must implement the following modular structure.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_PROD 50
#define MAX_LEN 100
#define MAX_NONTERM 26
#define MAX_TERM 50

/* ============================
   Data Structures
   ============================ */

/* Grammar Production */
typedef struct {
    char lhs;
    char rhs[MAX_LEN];
} Production;

/* Grammar Storage */
Production grammar[MAX_PROD];
int prodCount = 0;

/* FIRST and FOLLOW sets */
char FIRST[MAX_NONTERM][MAX_TERM];
char FOLLOW[MAX_NONTERM][MAX_TERM];

/* Parsing Table */
char parsingTable[MAX_NONTERM][MAX_TERM][MAX_LEN];

/* Stack for LL(1) parsing */
```

```
char stack[MAX_LEN];
int top = -1;

/* ============================
   Utility Functions
   ============================ */

void push(char c) {
    stack[++top] = c;
}

char pop() {
    return stack[top--];
}

void displayStack() {
    for(int i = 0; i <= top; i++)
        printf("%c", stack[i]);
}

/* ============================
   Function Prototypes
   ============================ */

void readGrammar();
void displayGrammar();

void removeLeftRecursion();
void leftFactoring();

void computeFirst();
void computeFirstOf(char symbol);

void computeFollow();
void computeFollowOf(char symbol);

void constructParsingTable();
void displayParsingTable();

void parseInputString();

/* ============================
   MAIN DRIVER
   ============================ */

int main() {

    readGrammar();
```

```
    printf("\nOriginal Grammar:\n");
    displayGrammar();

    removeLeftRecursion();
    printf("\nAfter Removing Left Recursion:\n");
    displayGrammar();

    leftFactoring();
    printf("\nAfter Left Factoring:\n");
    displayGrammar();

    computeFirst();
    computeFollow();

    constructParsingTable();
    displayParsingTable();

    parseInputString();

    return 0;
}

/* ============================
   Grammar Input
   ============================ */

void readGrammar() {

    int n;
    printf("Enter number of productions: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("Enter production (e.g. S->Sa|b): ");
        scanf("%s", grammar[i].rhs);
        grammar[i].lhs = grammar[i].rhs[0];
        prodCount++;
    }
}

void displayGrammar() {
    for(int i = 0; i < prodCount; i++)
        printf("%c -> %s\n", grammar[i].lhs, grammar[i].rhs + 3);
}

/* ============================
   Left Recursion Removal
   ============================ */
```

```
void removeLeftRecursion() {

    /* Students must:
       1. Detect productions of form A -> A
       2. Separate    and    parts
       3. Create new non-terminal A'
       4. Modify grammar accordingly
    */
}

/* ============================
   Left Factoring
   ============================ */

void leftFactoring() {

    /* Students must:
       1. Detect common prefixes
       2. Introduce new non-terminals
       3. Rewrite productions
    */
}

/* ============================
   FIRST Computation
   ============================ */

void computeFirst() {

    /* Initialize FIRST sets */

    /* For each non-terminal call computeFirstOf() */
}

void computeFirstOf(char symbol) {

    /* Recursive FIRST computation logic */
}

/* ============================
   FOLLOW Computation
   ============================ */

void computeFollow() {

    /* Initialize FOLLOW(startSymbol) with '$' */
    /* Call computeFollowOf() for each non-terminal */
}
```

```
void computeFollowOf(char symbol) {

    /* FOLLOW computation logic */
}

/* ============================
   Parsing Table Construction
   ============================ */

void constructParsingTable() {

    /* Use FIRST and FOLLOW to fill table */
    /* Detect conflicts if multiple entries occur */
}

void displayParsingTable() {

    printf("\nPredictive␣Parsing␣Table:\n");

    /* Display formatted matrix */
}

/* ============================
   LL(1) Parsing
   ============================ */

void parseInputString() {

    char input[MAX_LEN];
    int ip = 0;

    printf("\nEnter␣input␣string␣(end␣with␣$):␣");
    scanf("%s", input);

    push('$');
    push(grammar[0].lhs);

    printf("\nStack\tInput\tProduction\n");
    printf("-----------------------------------------\n");

    while(top != -1) {

        displayStack();
        printf("\t%s\t", &input[ip]);

        char stackTop = stack[top];
        char currentInput = input[ip];

        if(stackTop == currentInput) {
```

```
            pop();
            ip++;
            printf("Match␣%c\n", currentInput);
        }
        else if(isupper(stackTop)) {

            /* Lookup parsing table entry */

            printf("Apply␣production\n");

            /* Replace non-terminal on stack */
        }
        else {
            printf("Syntax␣Error\n");
            return;
        }

        if(stackTop == '$' && currentInput == '$')
            break;
    }

    printf("\nString␣Accepted\n");
}
```

10. Submission Requirements

Students must submit:

- Complete C source code
- Sample input grammars
- Sample parsing outputs
- Explanation of algorithms used

11. Evaluation Criteria

- Correct left recursion removal
- Correct left factoring
- Correct FIRST/FOLLOW computation
- Correct parsing table
- Proper parsing trace output
- Code quality and modular design

12. Learning Outcomes

After completing this lab, students will be able to:

- Automate grammar transformation

- Construct LL(1) parser generators
- Understand full pipeline of predictive parsing
- Implement compiler front-end components

Sample Input for Left Recursion Removal

```
Number of productions: 1
S->Sa|bSc|bSd|e
```

Expected Output

```
After Removing Left Recursion:

S  -> bScS'|bSdS'|eS'
S' -> aS'|epsilon
```

Sample Output After Left Factoring

```
After Left Factoring:

S  -> bSX|eS'
X␣␣->␣cS'|dS'
S' -> aS'|␣epsilon
```

Sample FIRST Set Output

```
FIRST Sets:

FIRST(S)  = { b, e }
FIRST(S')␣=␣{␣a,␣epsilon␣}
FIRST(X)␣␣=␣{␣c,␣d␣}
```

Sample FOLLOW Set Output

```
FOLLOW Sets:

FOLLOW(S)  = { $, c, d }
FOLLOW(S')␣=␣{␣$,␣c,␣d␣}
FOLLOW(X)␣␣=␣{␣$,␣c,␣d␣}
```

Sample Predictive Parsing Table

```
        b          e          a          c          d          $
---------------------------------------------------------------------------

S     S->bSX     S->eS'
S'                                   S'->aS'  S'->  ␣␣␣␣␣␣S'->       S
   '->epsilon
X␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣X->cS'    X->dS'
```

Sample Parsing Input

```
Input string: b e c a $
```

Expected Parsing Trace

```
Stack         Input         Production
--------------------------------------------

S$            b e c a $     S -> bSX
bSX$          b e c a $     Match b
SX$           e c a $       S -> eS'
eS'X$         e c a $       Match e
S'X$␣␣␣␣␣␣␣␣␣c␣a␣$␣␣␣␣␣␣␣␣␣S' -> epsilon
X$            c a $         X -> cS'
cS'$          c a $         Match c
S'$␣␣␣␣␣␣␣␣␣␣␣a␣$␣␣␣␣␣␣␣␣␣␣␣S' -> aS'
aS'$          a $           Match a
S'$␣␣␣␣␣␣␣␣␣␣␣$␣␣␣␣␣␣␣␣␣␣␣␣␣S' -> epsilon
$             $             Match $

String Accepted
```

Sample Error Case

```
Input string: b e d $

Parsing Trace:
...
Syntax Error at symbol d
```